

```
In [51]: # Import required Libraries.....
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# upload dataset and capturing a glimpse of data.....
std_performance = pd.read_csv("StudentsPerformance.csv")
std_performance.head()
```

```
Out[51]:
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

```
In [52]: std_performance.tail()
```

```
Out[52]:
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
995	female	group E	master's degree	standard	completed	88	99	95
996	male	group C	high school	free/reduced	none	62	55	55
997	female	group C	high school	free/reduced	completed	59	71	65
998	female	group D	some college	standard	completed	68	78	77
999	female	group D	some college	free/reduced	none	77	86	86

```
In [53]: std_performance.shape
```

```
Out[53]: (1000, 8)
```

In [54]: *# Knowing the data (Columns, datatype, Total values, Null values).....*
 std_performance.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   gender                                1000 non-null   object
1   race/ethnicity                        1000 non-null   object
2   parental level of education           1000 non-null   object
3   lunch                                 1000 non-null   object
4   test preparation course               1000 non-null   object
5   math score                           1000 non-null   int64
6   reading score                        1000 non-null   int64
7   writing score                         1000 non-null   int64
dtypes: int64(3), object(5)
memory usage: 62.6+ KB
```

In [55]: *# Collecting the Whole Statistical Summary of data.....*
 std_performance.describe()

Out[55]:

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000

In [56]: *# Checking for Null Values.....*
 std_performance.isnull()

Out[56]:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
995	False	False	False	False	False	False	False	False
996	False	False	False	False	False	False	False	False
997	False	False	False	False	False	False	False	False
998	False	False	False	False	False	False	False	False
999	False	False	False	False	False	False	False	False

1000 rows × 8 columns

```
In [57]: # Checking for Duplicate Values.....
std_performance.duplicated()
```

```
Out[57]: 0      False
1      False
2      False
3      False
4      False
...
995    False
996    False
997    False
998    False
999    False
Length: 1000, dtype: bool
```

```
In [58]: # Knowing Datatypes.....
std_performance.dtypes
```

```
Out[58]: gender                object
race/ethnicity                object
parental level of education    object
lunch                        object
test preparation course        object
math score                    int64
reading score                  int64
writing score                  int64
dtype: object
```

```
In [59]: std_performance["math score"] = std_performance["math score"].replace(100, 90)
```

```
In [60]: # Calculating Mean of All Scores(Math, Reading, Writing).....
std_performance["math score"].mean()
```

```
Out[60]: np.float64(66.019)
```

```
In [61]: std_performance["reading score"].mean()
```

```
Out[61]: np.float64(69.169)
```

```
In [62]: std_performance["writing score"].mean()
```

```
Out[62]: np.float64(68.054)
```

```
In [63]: # Calculating Minimum and Maximum Scores in every Subject.....
std_performance["math score"].max()
```

```
Out[63]: 99
```

```
In [64]: std_performance["math score"].min()
```

```
Out[64]: 0
```

```
In [65]: std_performance["reading score"].max()
```

```
Out[65]: 100
```

```
In [66]: std_performance["reading score"].min()
```

```
Out[66]: 17
```

```
In [67]: std_performance["writing score"].max()
```

```
Out[67]: 100
```

```
In [68]: std_performance["writing score"].min()
```

```
Out[68]: 10
```

```
In [69]: # Grouping By Gender(Boys and Girls).....
std_performance.groupby("gender")
```

```
Out[69]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000020667848C20>
```

```
In [70]: # Average Marks Scored by each Gender.....
std_performance.groupby("gender")["math score"].mean()
```

```
Out[70]: gender
female    63.575290
male      68.645228
Name: math score, dtype: float64
```

```
In [71]: std_performance.groupby("gender")["reading score"].mean()
```

```
Out[71]: gender
female    72.608108
male      65.473029
Name: reading score, dtype: float64
```

```
In [72]: std_performance.groupby("gender")["writing score"].mean()
```

```
Out[72]: gender
female    72.467181
male      63.311203
Name: writing score, dtype: float64
```

```
In [73]: # Grouping data by parental level, lunch,gender and test preparation.....
std_performance.groupby("parental level of education")
```

```
Out[73]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002066462CB00>
```

```
In [74]: std_performance.groupby("lunch")
```

```
Out[74]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002066600FE30>
```

```
In [75]: std_performance.groupby("test preparation course")
```

```
Out[75]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x00000206665658C0>
```

```
In [76]: # New Column Creation(Total Marks).....
std_performance["total score"] = std_performance["math score"] + std_performance["r
```

```
In [77]: std_performance.head()
```

```
Out[77]:
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	t sc
--	--------	----------------	-----------------------------------	-------	-------------------------------	---------------	------------------	------------------	---------

0	female	group B	bachelor's degree	standard	none	72	72	74	
---	--------	---------	----------------------	----------	------	----	----	----	--

1	female	group C	some college	standard	completed	69	90	88	
---	--------	---------	-----------------	----------	-----------	----	----	----	--

2	female	group B	master's degree	standard	none	90	95	93	
---	--------	---------	--------------------	----------	------	----	----	----	--

3	male	group A	associate's degree	free/reduced	none	47	57	44	
---	------	---------	-----------------------	--------------	------	----	----	----	--

4	male	group C	some college	standard	none	76	78	75	
---	------	---------	-----------------	----------	------	----	----	----	--



```
In [78]: # Another New Column Created(Percentage)
std_performance["percentage"] = (std_performance["total score"]/300)*100
```

```
In [79]: std_performance.head()
```

```
Out[79]:
```

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	t sc
0	female	group B	bachelor's degree	standard	none	72	72	74	
1	female	group C	some college	standard	completed	69	90	88	
2	female	group B	master's degree	standard	none	90	95	93	
3	male	group A	associate's degree	free/reduced	none	47	57	44	
4	male	group C	some college	standard	none	76	78	75	

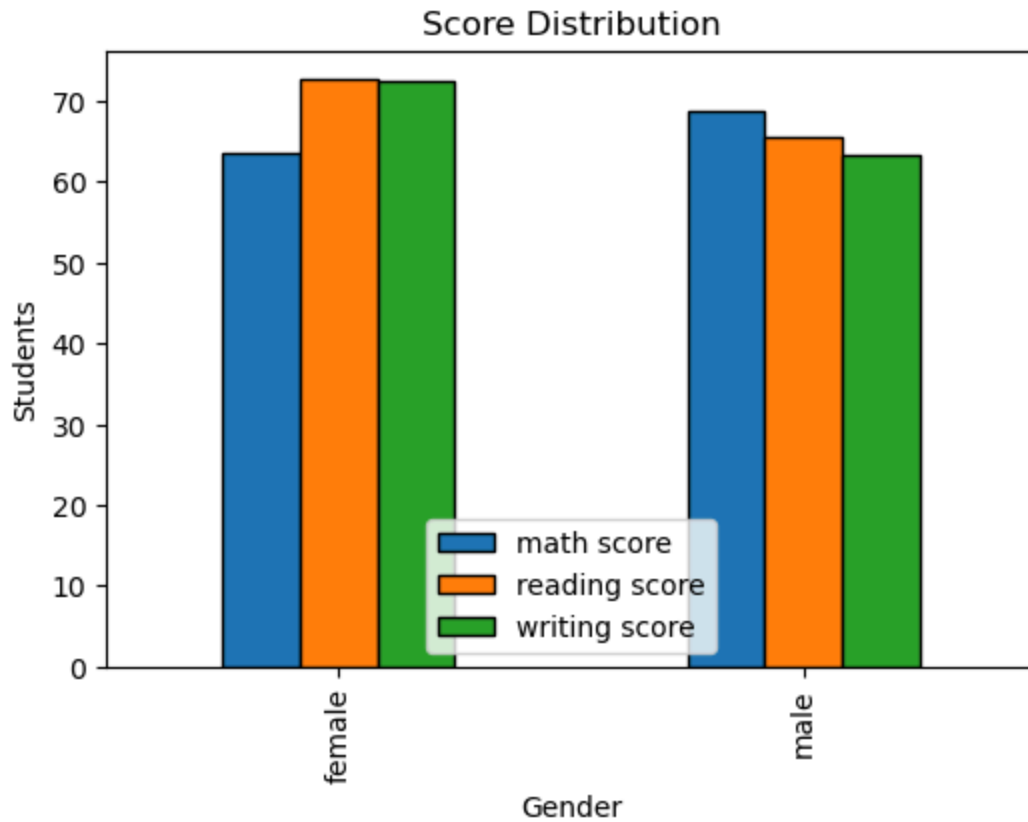
◀  ▶

```
In [80]: std_performance.groupby("gender")["percentage"].mean()
```

```
Out[80]: gender
female    69.550193
male      65.809820
Name: percentage, dtype: float64
```

```
In [81]: # Exploratory Data Analysis.....
# Visualization:-1.....
std_performance.groupby("gender")[["math score","reading score","writing score"]].m
plt.title("Score Distribution")
plt.xlabel("Gender")
plt.ylabel("Students")
```

```
Out[81]: Text(0, 0.5, 'Students')
```



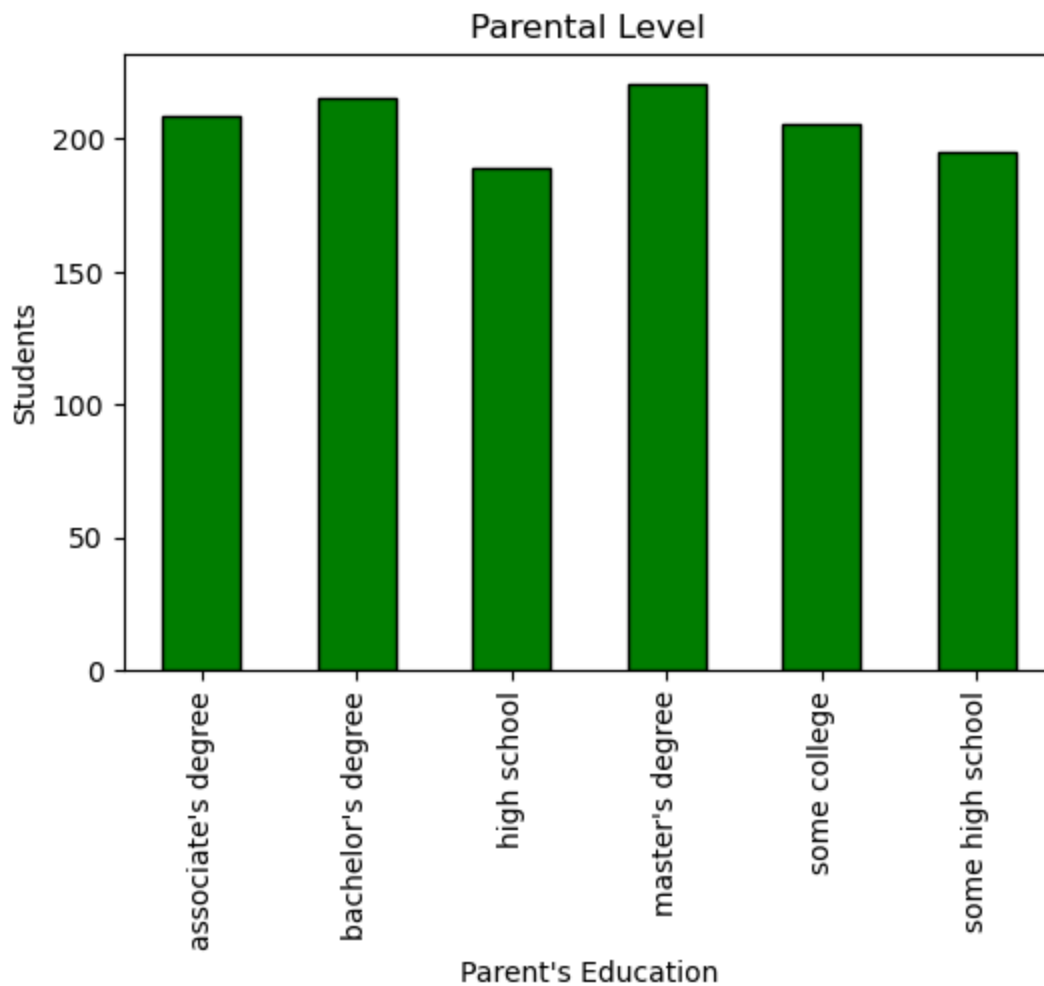
```
In [82]: # Visualization:-2.....  
std_performance.groupby("lunch")["total score"].mean().plot(figsize=(6,4),kind="bar")  
plt.title("Lunch Distribution")  
plt.xlabel("Lunch")  
plt.ylabel("Students")
```

```
Out[82]: Text(0, 0.5, 'Students')
```



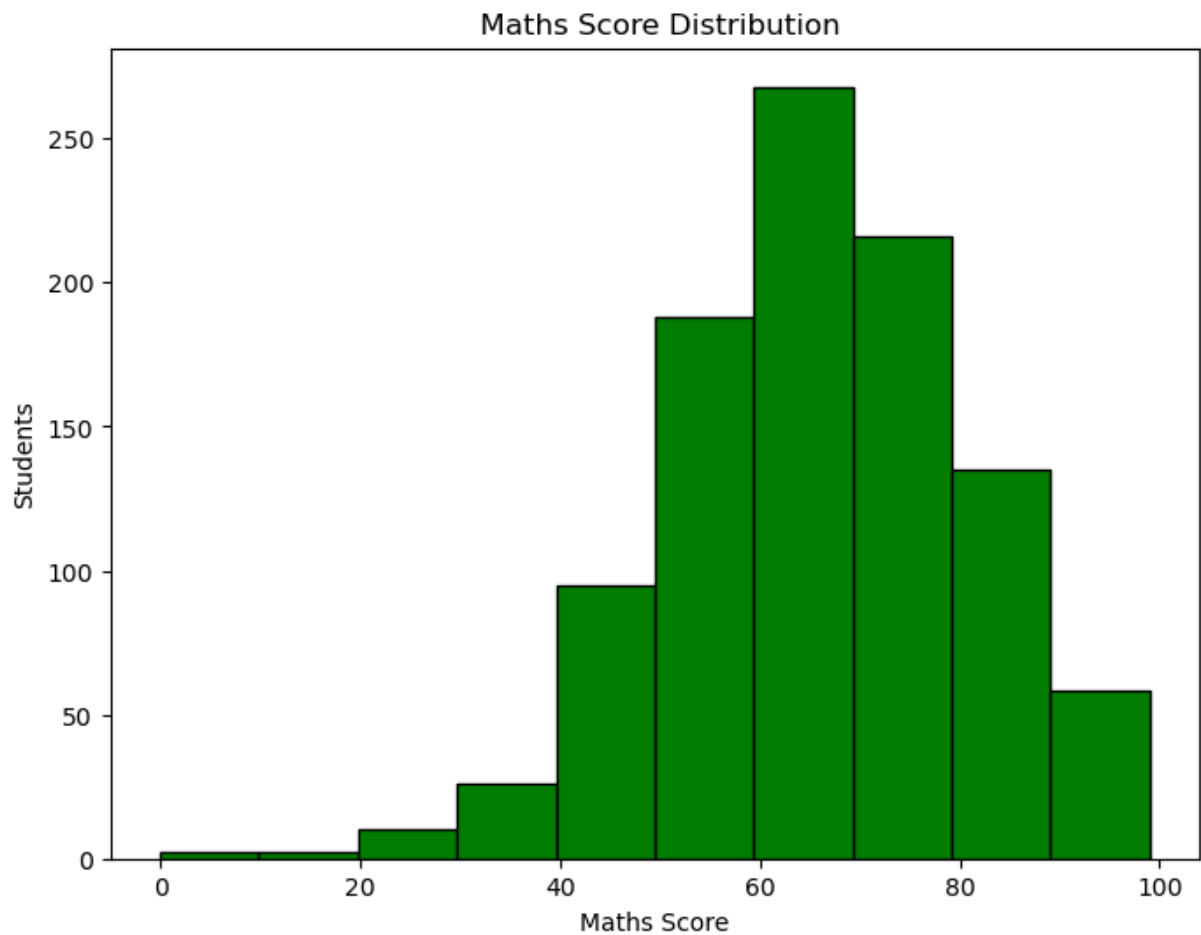
```
In [83]: # Visualization:-3.....
std_performance.groupby("parental level of education")["total score"].mean().plot(f
plt.title("Parental Level")
plt.xlabel("Parent's Education")
plt.ylabel("Students")
```

```
Out[83]: Text(0, 0.5, 'Students')
```



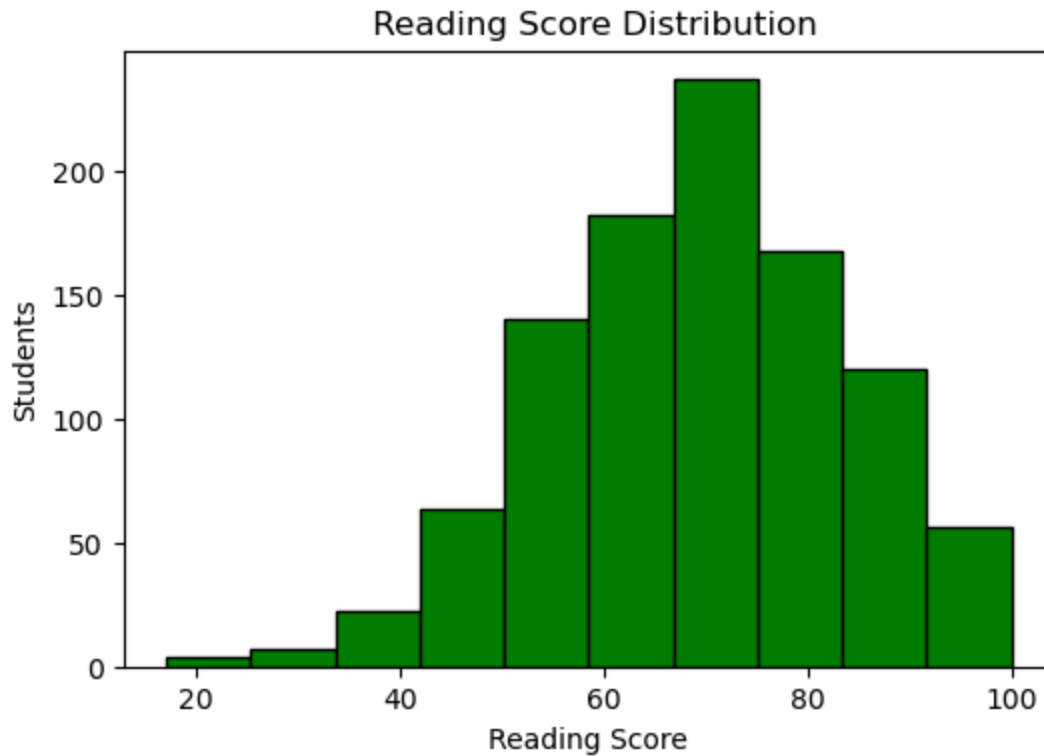
```
In [84]: # Visualization:-4.....
plt.figure(figsize=(8,6))
plt.hist(std_performance["math score"],
         edgecolor="black",
         color="green"
        )
plt.title(" Maths Score Distribution")
plt.xlabel("Maths Score")
plt.ylabel("Students")
```

```
Out[84]: Text(0, 0.5, 'Students')
```



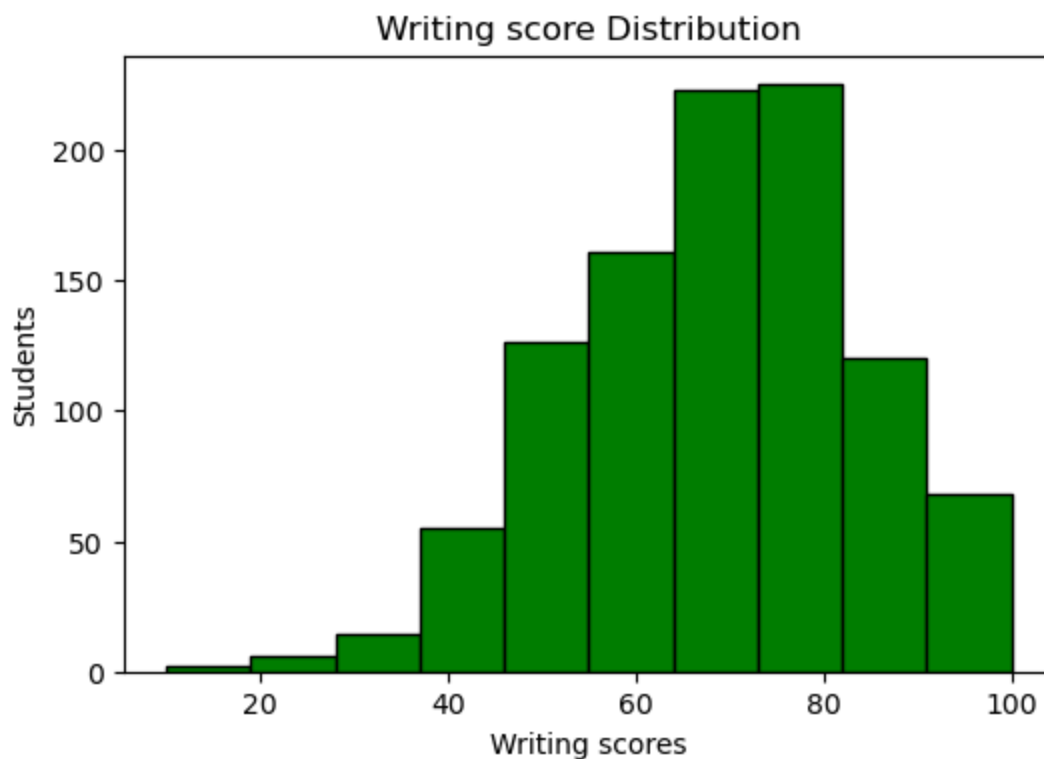
```
In [85]: # Visualization:-5.....  
plt.figure(figsize=(6,4))  
plt.hist(std_performance["reading score"],  
         edgecolor = "black",  
         color = "green"  
         )  
plt.title(" Reading Score Distribution")  
plt.xlabel("Reading Score")  
plt.ylabel("Students")
```

```
Out[85]: Text(0, 0.5, 'Students')
```



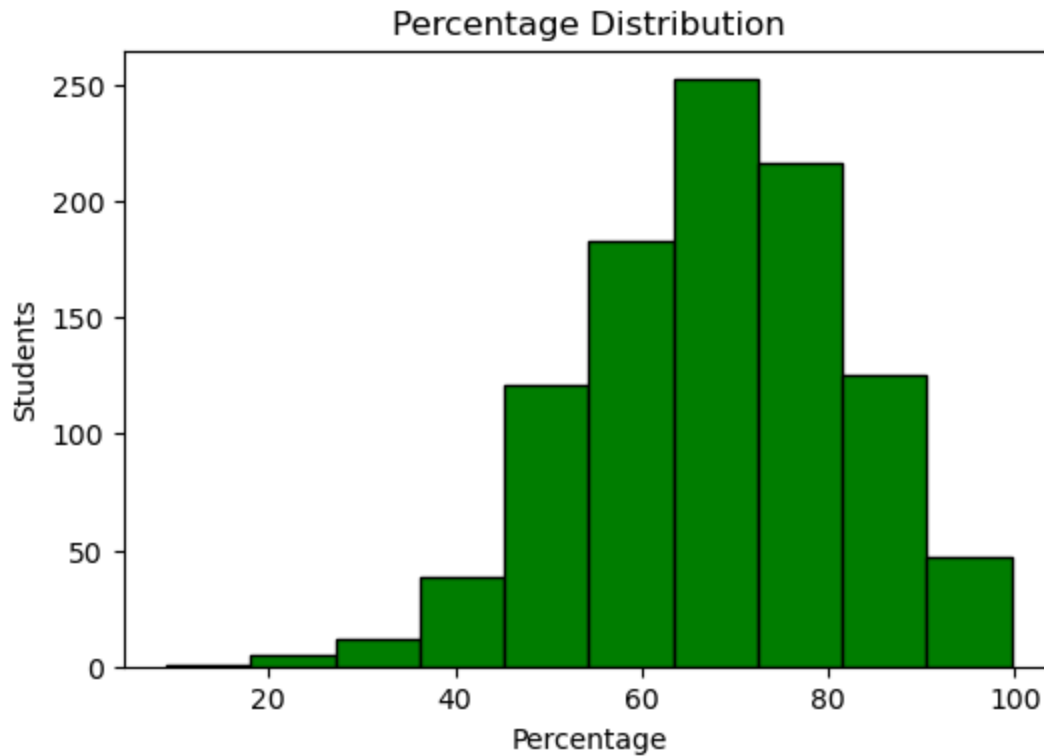
```
In [86]: # Visualization:-6.....
plt.figure(figsize=(6,4))
plt.hist(std_performance["writing score"],
        edgecolor="black",
        color="green"
        )
plt.title("Writing score Distribution")
plt.ylabel("Students")
plt.xlabel("Writing scores")
```

```
Out[86]: Text(0.5, 0, 'Writing scores')
```



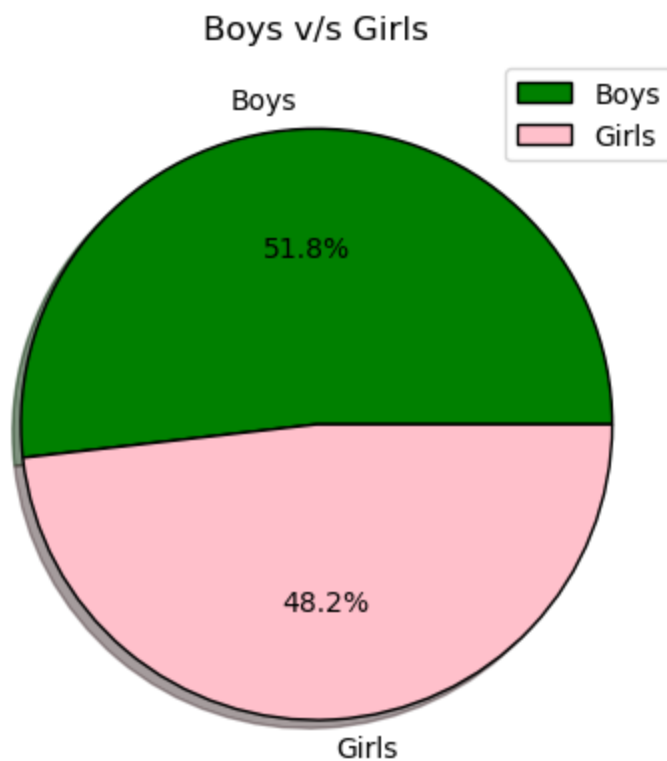
```
In [87]: # Visualization:-7.....
plt.figure(figsize=(6,4))
plt.hist(std_performance["percentage"],
         edgecolor="black",
         color="green",
         )
plt.title("Percentage Distribution")
plt.xlabel("Percentage")
plt.ylabel("Students")
```

```
Out[87]: Text(0, 0.5, 'Students')
```



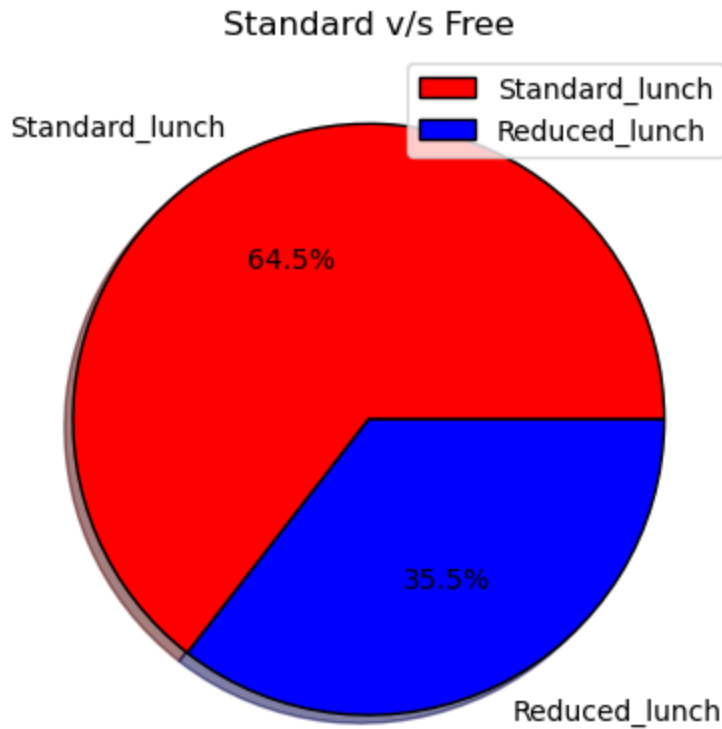
```
In [88]: # Visualization:-8.....  
plt.pie(std_performance["gender"].value_counts(),  
        labels=["Boys", "Girls"],  
        colors=["green", "pink"],  
        wedgeprops={'edgecolor': 'black'},  
        shadow=True,  
        autopct='%1.1f%%',  
        )  
plt.title("Boys v/s Girls")  
plt.legend()
```

```
Out[88]: <matplotlib.legend.Legend at 0x20664611810>
```



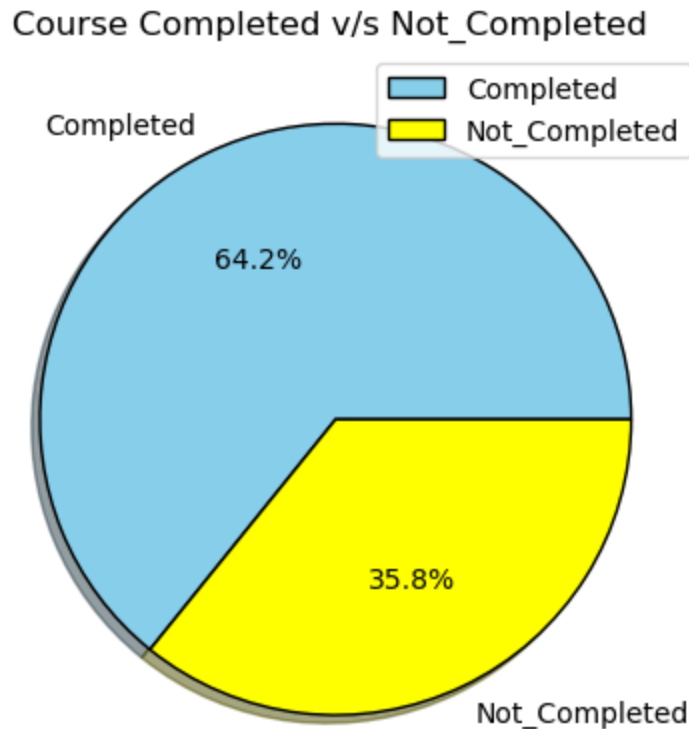
```
In [89]: # Visualization:-9.....
plt.pie(
    std_performance["lunch"].value_counts(),
    labels=["Standard_lunch", "Reduced_lunch"],
    colors=["red", "blue"],
    shadow = True,
    wedgeprops = {'edgecolor': 'black'},
    autopct = '%1.1f%%'
)
plt.title("Standard v/s Free")
plt.legend()
```

Out[89]: <matplotlib.legend.Legend at 0x206655bead0>



```
In [90]: # Visualization:-10.....
plt.pie(
    std_performance["test preparation course"].value_counts(),
    labels=["Completed", "Not_Completed"],
    colors=["skyblue", "yellow"],
    shadow=True,
    autopct='%1.1f%%',
    wedgeprops={'edgecolor':'black'}
)
plt.title("Course Completed v/s Not_Completed ")
plt.legend()
```

Out[90]: <matplotlib.legend.Legend at 0x206657c7d90>



```
In [91]: # Visualization:-11.....
boys = std_performance[std_performance["gender"] == "male"]
girls = std_performance[std_performance["gender"] == "female"]

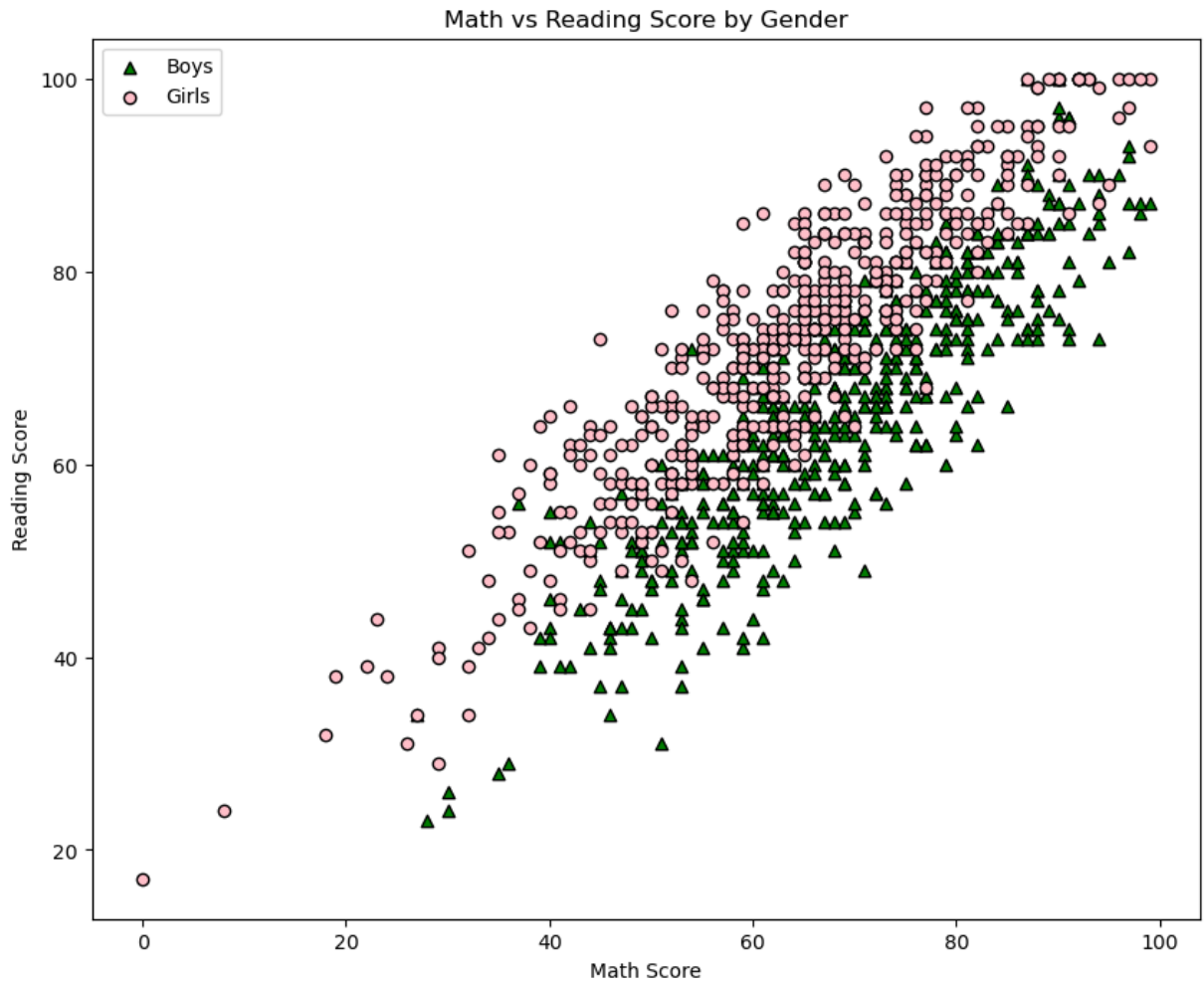
plt.figure(figsize=(10,8))

plt.scatter(
    boys["math score"],
    boys["reading score"],
    color="green",
    edgecolor="black",
    label="Boys",
    marker='^'
)

plt.scatter(
    girls["math score"],
    girls["reading score"],
    color="pink",
    edgecolor="black",
    label="Girls",
    marker='o'
)

plt.xlabel("Math Score")
plt.ylabel("Reading Score")
plt.title("Math vs Reading Score by Gender")
plt.legend()
```

Out[91]: <matplotlib.legend.Legend at 0x2066a9f4910>



```
In [92]: # Visualization:-12.....
boys = std_performance[std_performance["gender"] == "male"]
girls = std_performance[std_performance["gender"] == "female"]

plt.figure(figsize=(10,8))

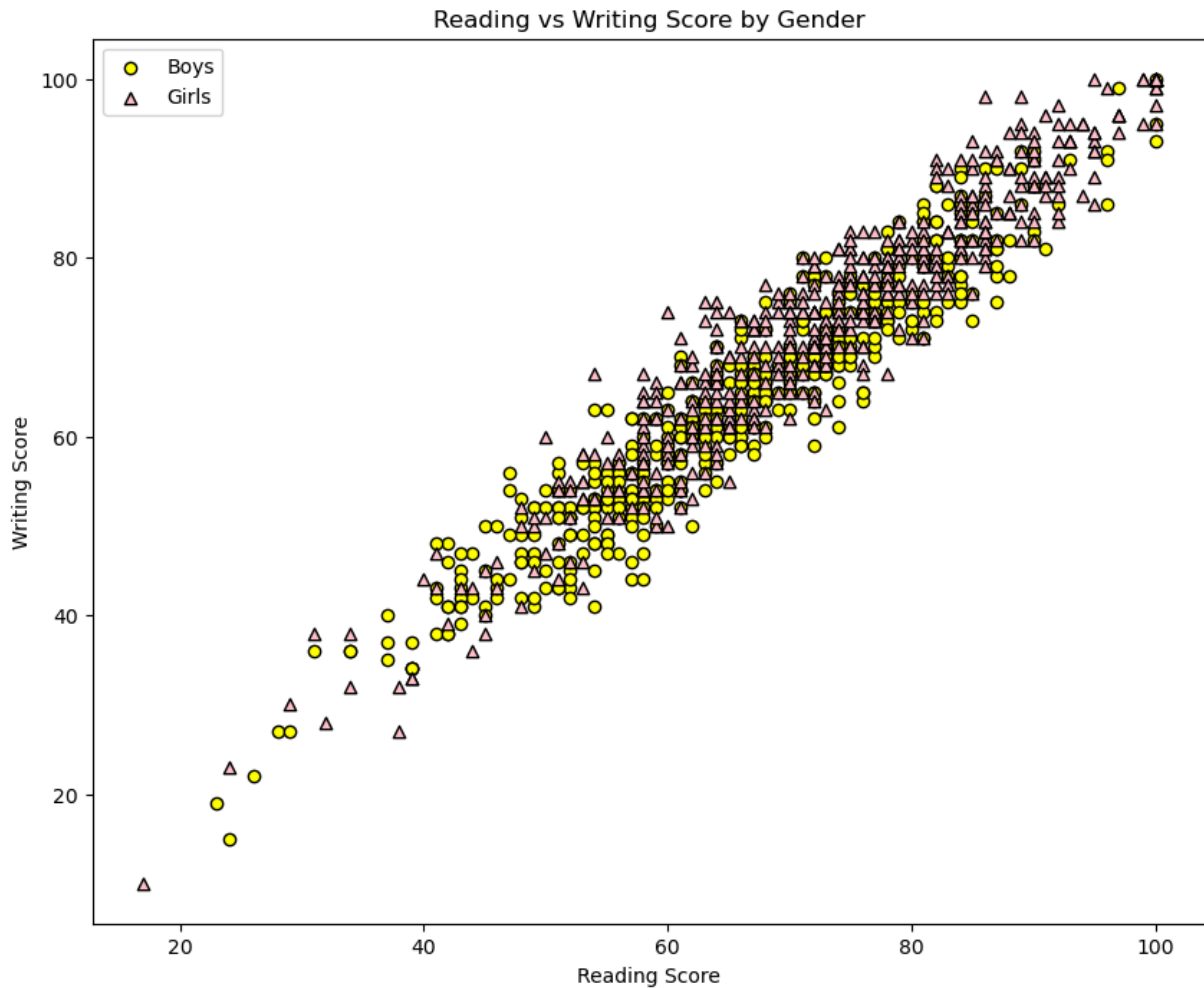
plt.scatter(
    boys["reading score"],
    boys["writing score"],
    color = "yellow",
    edgecolor = "black",
    marker='o',
    label="Boys"
)

plt.scatter(
    girls["reading score"],
    girls["writing score"],
    color = "pink",
    edgecolor = "black",
    marker = '^',
    label = "Girls"
)

plt.xlabel("Reading Score")
plt.ylabel("Writing Score")
```

```
plt.title("Reading vs Writing Score by Gender")
plt.legend()
```

Out[92]: <matplotlib.legend.Legend at 0x2066aa2b610>



```
In [93]: # Visualization:-13.....
boys = std_performance[std_performance["gender"] == "male"]
girls = std_performance[std_performance["gender"] == "female"]

plt.figure(figsize=(10,8))

plt.scatter(
    boys["math score"],
    boys["writing score"],
    color = "green",
    edgecolor = "black",
    marker = 'o',
    label = "Boys"
)
plt.scatter(
    girls["math score"],
    girls["writing score"],
    color = "yellow",
    edgecolor = "black",
    marker = '^',
    label = "Girls"
```

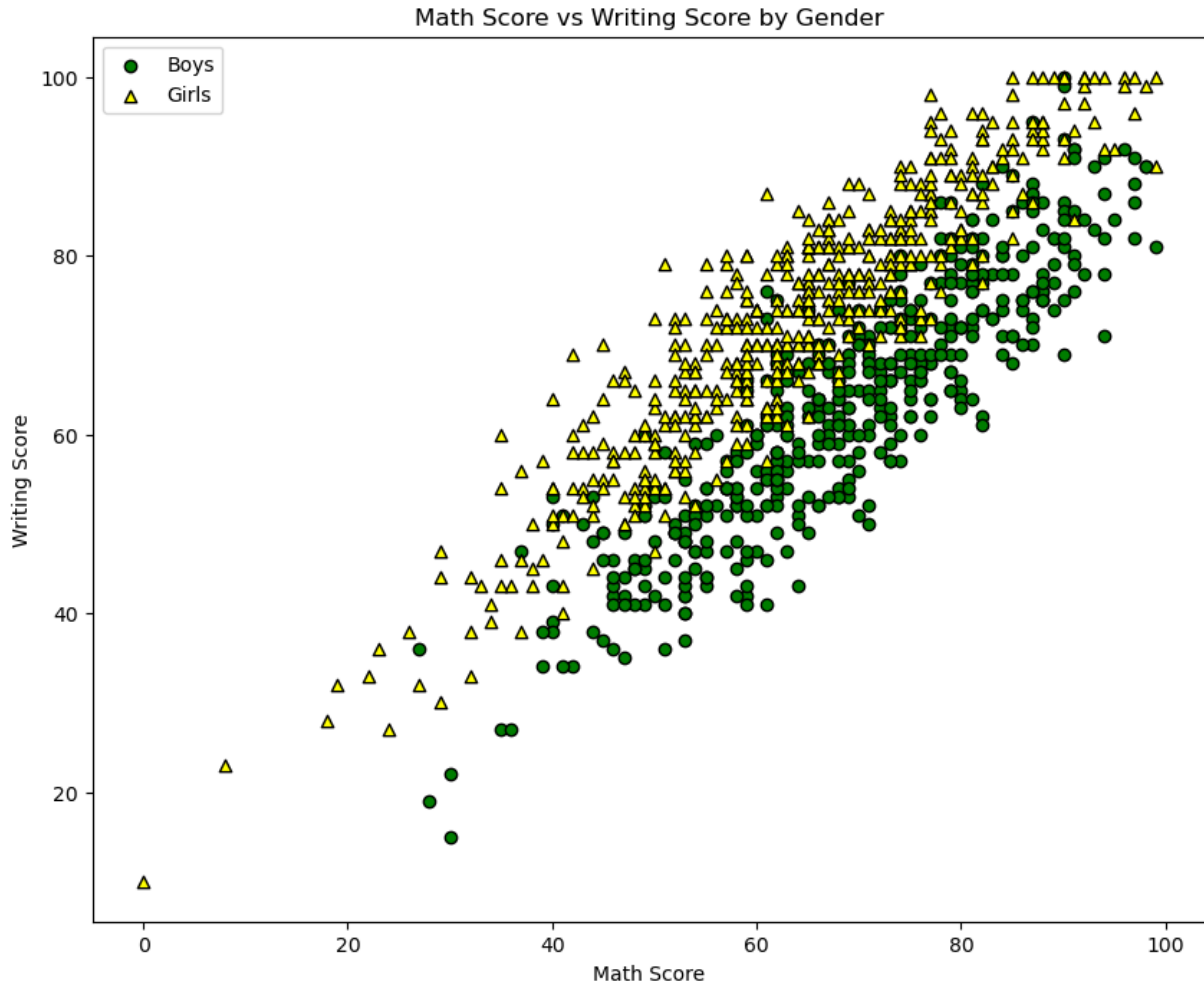
```

)

plt.title("Math Score vs Writing Score by Gender")
plt.xlabel("Math Score")
plt.ylabel("Writing Score")
plt.legend()

```

Out[93]: <matplotlib.legend.Legend at 0x2066a791bd0>

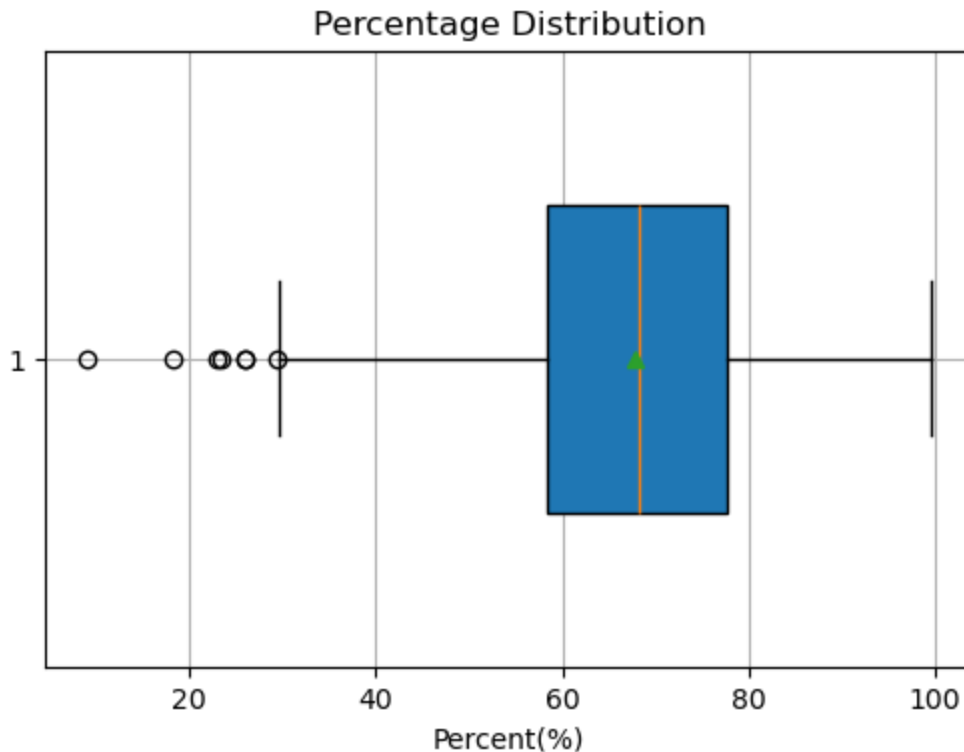


```

In [94]: # Visualization:-14.....
plt.figure(figsize=(6,4))
plt.boxplot(
    std_performance["percentage"],
    vert =False,
    patch_artist =True,
    showmeans =True,
    showfliers =True,
    widths =0.5
)

plt.title("Percentage Distribution")
plt.xlabel("Percent(%)")
plt.grid(True)

```



In [95]:

```
"""
```

Conclusion:

This project explored the Student Performance dataset using Python, Pandas, NumPy, and Matplotlib to identify patterns and factors associated with students' academic performance. Through data cleaning, exploratory data analysis (EDA), and data visualization, meaningful relationships between different variables and exam scores were identified.

The analysis indicated that factors such as test preparation course completion, parental level of education, and lunch type were associated with differences in student performance. It also highlighted a strong positive relationship among reading and writing scores and math scores, showing how these academic skills closely related.

Overall, this project strengthened my understanding of data preprocessing, exploratory data analysis, and data visualization techniques. It enhanced my ability to extract meaningful insights from real-world datasets and communicate them effectively using charts and statistical summaries.

This project serves as an important step in my Data Analytics and Machine Learning journey, providing a strong foundation for more advanced analytical and predictive modeling projects in the future.

```
"""
```

```
Out[95]: "\nConclusion:\n\nThis project explored the Student Performance dataset using Python, Pandas,\nNumPy, and Matplotlib to identify patterns and factors associated with\nstudents' academic performance. Through data cleaning, exploratory data\nanalysis (EDA), and data visualization, meaningful relationships between\ndifferent variables and exam scores were identified.\n\nThe analysis indicated that factors such as test preparation course completion,\nparental level of education, and lunch type were associated with differences\nin student performance. It also highlighted a strong positive relationship\namong reading and writing scores and math scores, showing how these academic skills are\nclosely related.\n\nOverall, this project strengthened my understanding of data preprocessing,\nexploratory data analysis, and data visualization techniques. It enhanced my\nability to extract meaningful insights from real-world datasets and communicate\nthem effectively using charts and statistical summaries.\n\nThis project serves as an important step in my Data Analytics and Machine\nLearning journey, providing a strong foundation for more advanced analytical\nand predictive modeling projects in the future.\n"
```